

End-to-End Containerized Data Engineering Architecture for Real-Time Micro-Mobility Telemetry

1st Yusuf Öksüzer

Artificial Intelligence and Data Eng. Dep.
İstanbul Technical University
İstanbul, Türkiye
oksuzer22@itu.edu.tr

2nd Yusuf Oğuz

Artificial Intelligence and Data Eng. Dep.
İstanbul Teknik Üniversitesi
İstanbul, Türkiye
oguzyu22@itu.edu.tr

Abstract—Real-time Internet of Things (IoT) data processing processes, especially in urban micro-mobility systems, increasingly require end-to-end data architectures capable of reliably managing high-frequency telemetry streams. This study presents a fully containerized data engineering pipeline developed to process synthetic location, battery, and hardware failure data from a simulated electric scooter fleet operating in the Şişli-Beşiktaş region. All components in the system are designed to boot up with a single Docker Compose command to provide a replicable and portable deployment model in a local environment. At the data generation layer, a custom Python simulator models 50 scooters and generates between 10,000 and 72,000 data points per hour in JSONL format containing realistic movement patterns, battery consumption curves, and intentionally injected anomalies (GPS loss, tipping, critical charge drop). Apache NiFi receives this stream in a decoupled form via a 'tail-file' mechanism; Structured telemetry logs are routed to PostgreSQL for billing and route analytics, while semi-structured hardware alerts and error codes are routed to Elasticsearch for anomaly querying. The Elasticsearch index is structured with 'geo_point' mapping to enable geo-queries and Kibana map visualizations. In the batch processing layer, Apache Airflow automates operational reporting by scheduling DAGs that calculate daily usage densities (hotspots) and revenue summaries. Kibana integrates all these data layers by providing an operational dashboard that visualizes breakdown alerts and scooter locations in real time. This developed system offers a modular and scalable reference architecture for similar smart city applications, with the capacity to process high-frequency sensor data with low latency, automatically classify heterogeneous data types, and orchestrate periodic analyses.

Index Terms—Data Engineering, IoT Telemetry, Containerization, Apache NiFi, Apache Airflow, Elasticsearch, Micro-Mobility

I. EXECUTIVE SUMMARY

This report outlines the design, implementation, and evaluation of a fully containerized data engineering architecture specifically designed for urban micro-mobility infrastructure. As electric scooter fleets become a primary means of short-distance urban transportation, the need for robust systems capable of handling high-volume telemetry data has increased significantly.

Our solution addresses this need by processing simulated data from a fleet of 50 active scooters operating on the Şişli-

Beşiktaş route, generating approximately 50 records per second. The entire architecture is orchestrated via Docker, featuring automated setup scripts (e.g., `nifi-setup`, `es-setup`) to ensure a zero-touch, repeatable deployment model without the need for manual host configurations.

To optimize data ingestion and eliminate routing bottlenecks, a dual-write architecture was implemented. A custom Python simulator directly writes structured telemetry to a PostgreSQL database for persistence, while simultaneously streaming JSONL payloads to a shared volume. Apache NiFi acts exclusively as a real-time anomaly router; utilizing a decoupled `TailFile` mechanism, it filters hardware faults and forwards them to an Elasticsearch cluster while intentionally dropping normal telemetry. Live spatial tracking of these anomalies via Kibana maps is enabled by configuring explicit `geo_point` mappings on Elasticsearch. Finally, Apache Airflow is integrated into the system to execute nightly batch jobs that calculate daily revenue and fleet utilization hotspots. The resulting system demonstrates a highly scalable, fault-tolerant approach to IoT data management, balancing high-frequency sensor streams with real-time operational monitoring and periodic batch analytics.

II. PROBLEM STATEMENT AND MOTIVATION

Managing a modern electric scooter fleet presents significant data engineering challenges due to the volume, velocity, and variety of telemetry data generated. Devices continuously broadcast GPS coordinates, battery status, and hardware conditions. When operating in high-density urban areas like the Şişli-Beşiktaş route, data delays or losses can lead to direct operational disruptions, such as the inability to collect vehicles with depleted batteries or the inability to respond immediately to hardware.

The fundamental problem is that traditional architectures, whether monolithic or batch-only, are insufficient for real-time IoT monitoring processes. If a scooter experiences a critical malfunction (e.g., tipping over or sudden GPS signal loss), the operations center needs to be immediately notified so that maintenance teams can be dispatched. In contrast, financial

metrics and usage intensity analyses do not require millisecond latency accuracy and are far more efficient when calculated in daily batches. Therefore, it is essential that the architecture supports a dual-path data pipeline that can route heterogeneous data to the most suitable storage environments according to instantaneous usage needs.

Our motivation in establishing this specific data pipeline is to bridge the gap between raw IoT sensor outputs and actionable operational intelligence. By engineering a decoupled system where data acquisition (NiFi), search/tracking (Elasticsearch/Kibana), and relational batch processing (PostgreSQL/Airflow) phases are containerized and isolated, we aim to provide a highly available, scalable infrastructure. This project aims to demonstrate how complex and high-frequency data streams can be reliably captured, parsed, and visualized to maintain the health and profitability of a micro-mobility fleet without overloading the infrastructure.

III. ARCHITECTURAL DESIGN

The architectural design of our project consists of four primary layers: data generation, ingestion, storage, and processing/visualization. To ensure continuous development and parallel collaboration, the entire architecture was designed as fully isolated microservices deployed via Docker.

A. Component and Data Flow Diagram

The data flow within the system is unidirectional and asynchronous. The process initiates with the Python-based simulator, which continuously writes synthetic scooter telemetry to a local directory in `.jsonl` (JSON Lines) format.

Apache NiFi serves as the primary backbone of the data pipeline. By employing a `TailFile` mechanism, NiFi remotely monitors and ingests this file. The raw data is parsed using built-in processors and divided into two distinct streams via content-based routing:

- **Stream 1 (Relational Storage):** All telemetry records are written directly to PostgreSQL by the simulator via `psycpg2`, bypassing NiFi entirely.
- **Stream 2 (Anomaly Routing):** Records carrying a non-empty `anomaly_type` field are forwarded by NiFi to the Elasticsearch cluster for real-time anomaly querying and map visualization. Normal records are intentionally dropped by the router.

At the terminals of this split pipeline are business intelligence and monitoring tools. Kibana connects to Elasticsearch to visualize real-time alerts and geographical scooter locations on a map. Concurrently, Apache Airflow reads persistent data from PostgreSQL to generate batch analytical summaries, such as daily hotspot and revenue reports.

B. Rationale for Design Decisions

The architectural decisions are fundamentally driven by the principles of system decoupling and polyglot persistence.

Avoiding Direct Database Ingestion: Directly writing the 72,000 records generated per hour by the simulator into PostgreSQL or Elasticsearch would risk significant data loss

in the event of a connection failure. Positioning Apache NiFi as an intermediary buffer and router completely isolates the data source from the storage destinations. NiFi's `TailFile` mechanism maintains internal state tracking; therefore, even during a system crash, it remembers the exact byte offset of the file, reducing the risk of data loss to zero upon recovery.

Dual Storage Strategy (PostgreSQL and Elasticsearch):

Rather than consolidating all data into a single repository, storage engines were selected based on the inherent nature of the data. Operations requiring strict schemas and ACID compliance, such as billing and daily revenue calculations executed by Airflow, are allocated to PostgreSQL. Conversely, relational databases are inefficient at performing full-text lookups within text-based error codes or clustering thousands of coordinates on a map in milliseconds (geo-queries). Thus, the burden of anomaly querying and spatial tracking is delegated to Elasticsearch, which executes these tasks with significantly higher performance due to its inverted index architecture.

IV. TOOLS USED AND RATIONALE FOR THEIR SELECTION

The technology stack used in this project was carefully selected to meet the containerization and high-throughput requirements specified in the guidelines. Each component was positioned to solve a specific bottleneck in the data flow by comparing it to popular alternatives.

A. Data Generation and Orchestration

Python Simulator (vs a static dataset): Instead of using static Kaggle datasets to test the system, a custom Python-based simulator was developed to model a 50-scooter fleet generating approximately 50 records per second. This approach enabled the deterministic injection of hardware anomalies (e.g., GPS loss, critical battery) and implemented a dual-write strategy to optimize ingestion throughput.

Docker and Docker Compose (vs virtual machines):

To ensure absolute system portability, Docker was chosen over traditional virtual machines (VMs) or local installations. Thanks to Docker Compose, the entire architecture can be deployed using a single YAML file. Furthermore, we integrated ephemeral setup containers (`nifi-setup`, `es-setup`, `kibana-setup`) to automate infrastructure initialization, enabling a zero-touch, repeatable deployment model.

B. Ingestion & Event Routing

Apache NiFi (vs Apache Kafka or Logstash): While Kafka is an industry-standard high-throughput message queue, it requires additional consumer microservices to process data. In our updated dual-write architecture, NiFi acts specifically as an anomaly filter rather than a universal router. Its visual interface and `RouteOnAttribute` processor allow it to elegantly isolate hardware faults for Elasticsearch while intentionally dropping normal telemetry (which is already secured in PostgreSQL). Furthermore, its `TailFile` processor offers strict state management, ensuring no critical alerts are lost during container restarts.

C. Storage Layer

PostgreSQL (vs MongoDB or MySQL): PostgreSQL is chosen as the primary relational backbone for data with strict schemas. By having the Python simulator write standard telemetry directly to PostgreSQL, we bypassed potential NiFi routing bottlenecks. While NoSQL solutions like MongoDB offer flexibility, the complex SQL "JOIN" and "GROUP BY" operations required by Airflow for daily revenue or grid hotspot calculations are significantly more performant in PostgreSQL.

Elasticsearch (vs relational alternatives): Elasticsearch was utilized specifically for the `scooter-alerts` index to handle hardware errors and real-time location data. While searching for text-based error codes in tens of thousands of rows in a standard database can take seconds, Elasticsearch accomplishes this in milliseconds thanks to its inverted index architecture. Additionally, its native support for the `geo_point` data type makes Kibana spatial queries highly efficient.

D. Processing and Visualisation

Apache Airflow (vs Cron or Luigi): Airflow manages the batch processing layer. While simple Linux Cron jobs are insufficient for managing task dependencies (e.g., ensuring trip calculations finish before revenue aggregations), Airflow increases operational reliability by providing automatic retry mechanisms, DAG scheduling (running nightly at 02:00 UTC), and an observable web interface.

Kibana (vs Grafana): For real-time anomaly visualization, Kibana was preferred over Grafana due to its direct, native integration with Elasticsearch. Specifically, the Kibana Maps module offers unparalleled ease of use in converting `geo_point` data into dynamic map layers.

V. IMPLEMENTATION DETAILS

This section details the technical implementation processes of the pipeline, particularly the automation strategies, statistical models in data generation, and advanced configurations in the NiFi/Airflow layers.

A. Automated Infrastructure Setup and Healthcheck Chain

To fulfill the "one-command setup" requirement in the instructions, the system is built on a strict container dependency chain (healthcheck). Containers don't start up randomly; instead, they wait for each other to reach a "healthy" state.

- **Database priority:** The 'simulator' service will not start until the 'PostgreSQL' container is fully ready. Temporary containers like 'nifi-setup' and 'es-setup' are activated as soon as the main services' APIs begin responding, perform the necessary configurations, and automatically shut down once their tasks are complete.

B. Data Generation and Probabilistic Modeling

The Python-based simulator models a fleet of 50 scooters, generating 50 records per second. To ensure a realistic data flow, the following statistical probabilities are run in each second cycle:

- **Drive Start:** A free scooter has a 5% chance of starting a ride every second.
- **Drive finish:** A device in motion is designed with a 3% probability of its journey ending.
- **Anomaly Injection:** To test system robustness, an anomaly (*topple*, *critical_battery*, *gps_loss*, *motor_fault*) is generated with a 4% probability in each recording cycle.

C. NiFi flow control and REST API Integration

Apache NiFi version 1.25.0 employs a unique method via the 'nifi-setup' container to overcome inconsistencies in importing XML templates. Instead of using pre-made templates, the system programmatically creates all processors and connections individually through HTTP requests via the NiFi REST API.

In the real-time anomaly parsing process, the following NiFi Expression Language (NEL) code was used on the `RouteOnAttribute` operator:

```
${anomaly_type:isEmpty():not() }
```

This code checks if the 'anomaly_type' field in the JSON payload is filled, redirecting only erroneous records to Elasticsearch and discarding empty (normal) records, thus preventing unnecessary load on the system.

D. Elasticsearch and Geo-Point Configuration

To enable the visualization of spatial analyses on Kibana Maps, the `scooter-alerts` index was manually configured before the data stream. Instead of dynamic mapping, the location field was explicitly defined as the `geo_point` data type. This ensures that latitude and longitude data are processed not just as numerical values, but as searchable coordinates on the map.

E. Airflow DAG Structure and Parallel Processing

The DAG named `scooter_daily_summary` on Apache Airflow is triggered every night at 02:00 UTC. A parallel processing architecture has been established between tasks to increase efficiency:

- first, the `compute_trip_summaries` task runs and identifies completed drives.
- When this task is successfully completed, `compute_daily_hotspots` and `compute_daily_revenue` tasks are executed simultaneously (in parallel), without waiting for each other.

This structure minimizes the total execution time of heavy analytical queries, thus reducing the risk of database lockups.

VI. EVALUATION

To verify that the pipeline behaves correctly under sustained load, we ran a series of measurements covering ingestion throughput, routing accuracy, data quality, and batch processing. All figures below come from a single uninterrupted session rather than cherry-picked runs.

A. Throughput and Volume Metrics

We left the stack running for roughly 7.5 hours (12:01 to 19:49 UTC, 10 May 2026) without touching it. At the end of the session we queried PostgreSQL directly and got the following numbers:

- Total records ingested: 680,750
- Total anomaly records: 27,109 (3.98%)
- Session duration: ~ 7.5 hours
- Effective throughput: $\sim 25,200$ records/minute

The 3.98% anomaly share is essentially identical to the 4% target we hardcoded into the simulator, which tells us the probabilistic injection logic is holding up correctly at scale, it is not drifting over long runs.

B. NiFi Routing Accuracy

The whole point of the NiFi layer is to forward only anomalies to Elasticsearch and drop everything else. To check this, we compared the document count in the `scooter-alerts` index against the anomaly row count in PostgreSQL. Both came out at 27,109. That match means two things:

- 1) No normal record slipped through the `RouteOnAttribute` filter.
- 2) No anomaly was silently discarded, every one of them made it to Elasticsearch.

The routing expression `${anomaly_type:isEmpty():not() }` turned out to be the right call here; earlier attempts with `isNull()` failed because NiFi converts JSON nulls to empty strings rather than actual null attributes.

C. Data Quality Checks

We looked at three dimensions:

- 1) **Completeness:** All 680,750 rows carry the mandatory fields (`event_id`, `scooter_id`, `ts`, `speed_kmh`, `battery_pct`) with no NULLs in columns that are not supposed to have them.
- 2) **Consistency:** `anomaly_type` and `fault_code` are either both NULL (a normal ride record) or both filled (an anomaly event). We spot-checked several hundred rows and found no mismatched pairs.
- 3) **Geo-validity:** Scooters that lost GPS signal have `gps_valid = FALSE` and a NULL location field. The Kibana map layer only renders documents that carry a proper `geo_point` value, so those records are automatically excluded from the map without any extra filtering.

D. Service Health and Startup Reliability

On a standard development machine (16 GB RAM, 8-core CPU), all 11 containers were up and healthy within about 4 minutes of running `docker compose up --build` for the first time. The healthcheck-based `depends_on` chain in the Compose file did its job: no service tried to connect to a dependency that was not yet ready. Figure 1 shows the startup output confirming every container reached a running state in the correct order.

```
(*) start 12/12
✓Container scooter-pipeline-postgres-1    Healthy      10.7s
✓Container scooter-pipeline-elasticsearch-1 Healthy      10.8s
✓Container scooter-pipeline-nifi-1        Started      0.35s
✓Container scooter-pipeline-nifi-setup-1   Started      0.2s
✓Container scooter-pipeline-simulator-1    Started      1.63s
✓Container scooter-pipeline-airflow-int-1   Exited       15.75s
✓Container scooter-pipeline-gpsd-n-1       Started      1.7s
✓Container scooter-pipeline-airflow-webserver-1 Started      0.45s
✓Container scooter-pipeline-airflow-scheduler-1 Started    0.44s
✓Container scooter-pipeline-kibana-1       Healthy      14.5s
✓Container scooter-pipeline-es-setup-1     Started      0.4s
```

Fig. 1. All 11 containers running after `docker compose up --build`.

E. Airflow DAG Correctness

Because the DAG is scheduled for 02:00 UTC, we triggered it manually during the session to catch any issues before the first real run. All four tasks finished without errors:

- `ensure_postgres_connection` — database was reachable
- `compute_trip_summaries` — `public.trips` populated
- `compute_daily_hotspots` — `public.daily_hotspots` populated (ran in parallel)
- `compute_daily_revenue` — `public.daily_revenue` populated (ran in parallel)

The full DAG completed in under 2 minutes for the 7.5 hour dataset. For a nightly job that has an entire night to finish, that is a comfortable margin.

VII. LIMITATIONS AND FUTURE WORK

This section discusses the current limitations of the developed data engineering pipeline and the targeted future improvements for scaled-up to industrial production.

A. Current Limitations

The project has some technical limitations due to its scope and the use of local resources:

- **single Node Infrastructure:** The current system operates via Docker containers on a single physical machine. While this might be sufficient for a load of 50 records per second, it can lead to vertical and horizontal scaling bottlenecks under the load created by a real-world fleet of tens of thousands of scooters.
- **Simulation Data and External Factors:** Although the data was generated by a high-accuracy simulator, external variables affecting driving performance and battery life, such as weather conditions, traffic density, and terrain gradient, were not fully incorporated into the model.
- **Data Archiving (Cold Storage):** The system stores all telemetry data on PostgreSQL and Elasticsearch. As

data volume increases, a mechanism for migrating older data to "cold storage" tiers like Amazon S3 or Google Cloud Storage for cost-effectiveness has not yet been established.

B. Future Work

The planned improvements to increase the operational efficiency of the system are as follows:

- **Cloud Migration:** Migrating the existing Docker architecture to AWS (Amazon Web Services) or GCP (Google Cloud Platform) and managing it with Kubernetes (EKS/GKE) will make the system fully scalable and highly available.
- **Predictive Maintenance:** Machine learning models can be trained using historical anomaly and battery data accumulated in PostgreSQL. Thanks to this models, the probability of a scooter malfunctioning can be predicted before it happens, thus reducing operating costs.
- **Dynamic Pricing:** The density (hotspot) reports generated by Airflow can be combined with a real-time API, establishing the infrastructure for a dynamic pricing model in high-demand areas.
- **Advanced Security Layer:** Data security will be brought to industry standards by using SSL/TLS certificates in inter-service communication and strengthening user-based access control (RBAC) mechanisms.

VIII. AI USAGE DECLARATION

We occasionally used LLM tools while writing documentation, debugging SQL queries, and improving wording in English. However, the system architecture, implementation decisions, and testing procedures were designed and validated manually by the team.

The main areas where AI tools were included in the project are listed below:

- **Academic Reporting and Translation:** Language models were used to support the structuring of the project report sections in an academic manner, the translation of Turkish technical descriptions into English according to international terminology, and the improvement of text fluency.
- **Code and Configuration Support:** Artificial intelligence was used for debugging in the stages of establishing probabilistic models (probability distribution) in the Python simulator, designing the NiFi Expression Language (NEL) logic, and optimizing complex SQL queries (JOIN and GROUP BY).
- **Architectural Documentation:** Structural support was used in the processes of expressing the data flow in the system (dual-write strategy, decoupling principles, etc.) in a technically correct language and justifying design decisions.

IX. INDIVIDUAL CONTRIBUTION TABLE

This section summarizes the primary tasks and responsibilities undertaken by the team members during the development and documentation phases of the project. The project

was completed through active collaboration and continuous coordination between both members.

Name Surname	Student ID	Primary Contributions and Responsibilities
Yusuf Oğuz	150220322	Design of the system architecture, establishment of the Docker container hierarchy, NiFi REST API integration, Python simulator logic, and development of Airflow DAGs.
Yusuf Öksüzler	150220334	Writing the academic report, documentation of the system architecture, execution of performance tests (throughput/latency), data quality controls, and analysis of design decisions.

TABLE I
TEAM MEMBERS AND TASK DISTRIBUTION

A. Contribution Summary

- **Architecture and Development:** End-to-end orchestration and optimization via Dockerized microservices and automated setup scripts were led by Yusuf Oğuz.
- **Analysis and Reporting:** The academic presentation of the system within the framework of engineering principles (decoupling, polyglot persistence) and the formulation of testing strategies were led by Yusuf Öksüzler.
- **Final Review:** Both members performed cross-reviews of each other's outputs (code and report) to ensure overall system integrity and technical accuracy.

REFERENCES

- [1] Apache Software Foundation, "Apache NiFi Documentation," 2024. [Online]. Available: <https://nifi.apache.org/docs.html>
- [2] Elastic, "Elasticsearch Guide," 2024. [Online]. Available: <https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html>
- [3] Apache Software Foundation, "Apache Airflow Documentation," 2024. [Online]. Available: <https://airflow.apache.org/docs/>
- [4] PostgreSQL Global Development Group, "PostgreSQL 15 Documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs/15/>
- [5] Docker Inc., "Docker Compose Overview," 2024. [Online]. Available: <https://docs.docker.com/compose/>